

Chapter 1

L5 - Access Methods: Hashing and B⁺-Trees

1.1 Where access methods fit

The DBMS execution engine reads and writes **pages**. To avoid scanning every page for every query, the DBMS uses **access methods**: physical data structures that help it find the right page or record quickly. At this layer, two large families dominate:

- **tree-based structures**: ordered access paths,
- **hash-based structures**: unordered access paths.

This difference is the key idea of the lecture:

- if the workload needs mainly **equality search**, hashing is often attractive,
- if the workload needs **equality and range search**, ordered trees are the standard choice.

1.2 How an index points to table data

Data entry k^* . The information stored in the index for search-key value k . It tells the DBMS how to reach the matching table record or records.

There are three classic representations for a data entry:

Three alternatives for index data entries

Alt.	Form of entry	Meaning
1	actual data record with key k	The index entry is the record itself. The index and the data file are physically the same structure.
2	$\langle k, \text{RID} \rangle$	The index stores the key and one pointer to one matching record.
3	$\langle k, \text{list of RIDs} \rangle$	The index stores one key and a list of pointers to all matching records.

The important point is that this choice is **orthogonal** to the indexing technique:

- a **hash index** can use Alternative 1, 2, or 3,
- a **tree index** can also use Alternative 1, 2, or 3.

1.2.1 Reading the three alternatives

- **Alternative 1:** the leaf level stores the full records. Very fast because no extra jump from index entry to data record is needed.
- **Alternative 2:** one pair $\langle k, \text{RID} \rangle$ per matching record. Simple, works well even when the search key is not unique.
- **Alternative 3:** one key with a full RID list. Saves space when many records share the same key value.

1.2.2 Why Alternatives 2 and 3 are often preferred

Suppose an employee file is itself hashed on **age**. Then the buckets store the **actual employee records**: this is Alternative 1.

Now suppose we also want an index on **salary**. We usually do *not* want to duplicate the full records inside a second structure. Instead, we create another access path that stores either:

- $\langle \text{salary}, \text{RID} \rangle$ pairs, or
- $\langle \text{salary}, \text{list of RIDs} \rangle$ entries.

This leads to the standard trade-offs:

- Alternatives 2 and 3 are easier to maintain than Alternative 1.
- If a file has several indexes, only **one** of them can use Alternative 1, because the data records can only be physically organized one way at a time.
- Alternative 3 is usually more compact than Alternative 2 when many duplicates exist.
- The downside of Alternative 3 is that entries become **variable-sized**, even when the search key has fixed length. If one key has a very long RID list, that entry may span several pages.

1.3 Hash indexes versus tree indexes

Hashing and trees solve different problems.

Hash-based index. Uses a hash function $h(k)$ to send a search-key value to one bucket. Because the buckets are not ordered by key value, hashing is best for equality search.

Tree-based index. Keeps entries in sorted search-key order. In practice, the standard disk-oriented tree structure is the B⁺-tree.

Hash indexes and tree indexes

Structure	Best at	Main limitation
Hash index	equality predicates such as age = 44	no natural order, so range predicates are poor
B ⁺ -tree index	equality, range, ordered scans, sequential access	usually a little more overhead for pure equality lookups

1.3.1 Why range search needs order

Consider the query:

```
SELECT *
FROM Student
WHERE gpa > 3.0
```

If the data file were physically sorted on **gpa**, the DBMS could:

- do a binary search to find the first matching student,
- then scan forward to collect the rest.

The problem is that keeping the *whole data file* sorted is expensive: insertions and deletions constantly disturb the order.

The simple idea is therefore:

- keep a smaller **index file** in sorted order,
- search the index first,
- then use it to reach the data pages.

This is the basic intuition behind the B^+ -tree: keep the *index* ordered, not the whole relation.

1.4 B^+ -trees

B^+ -tree. A self-balancing, ordered tree structure optimized for page-based storage. Search, insertion, deletion, and sequential access all work efficiently, typically in $O(\log_F N)$, where F is the fanout and N is the number of leaf nodes.

Fanout. The maximum number of child pointers an internal node can hold. Large fanout is what makes disk-based trees shallow.

1.4.1 Why B^+ -trees are so widely used

B^+ -trees and their variants are the preferred tree structures for external storage because they match the way disks and SSDs work:

- one node is typically one **page**,
- each internal node stores many separator keys and child pointers,
- high fanout means very small tree height,
- all leaves are linked in sorted order, which makes range scans efficient.

The broader family contains several variants, such as **B-tree**, B^+ -tree, **B*-tree**, **B-link tree**, and **B ϵ -tree**. In this lecture, the main focus is the B^+ -tree.

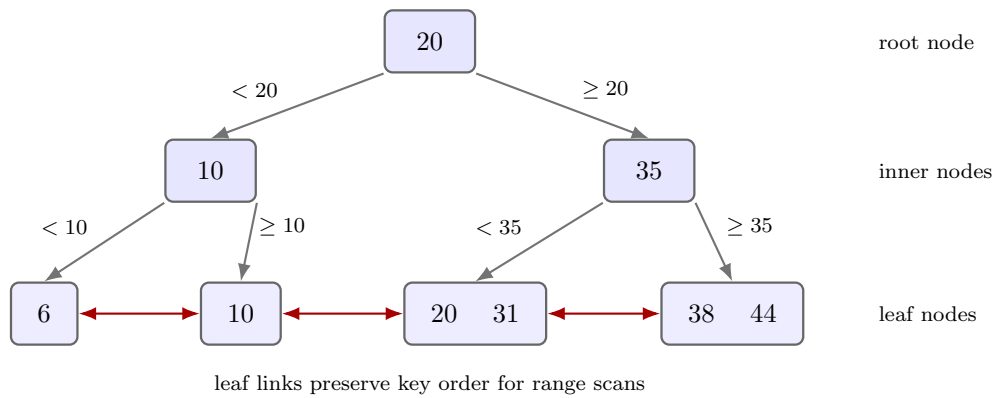
1.4.2 Core properties

A B^+ -tree is an n -way search tree with the following properties:

- nodes come in three roles: **root**, **inner**, and **leaf**,
- the tree is **perfectly balanced**: every leaf is at the same depth,
- every node except the root is at least **half full**,
- an inner node has between $\lceil n/2 \rceil$ and n children,
- if an inner node has k children, it has $k - 1$ separator keys,
- a leaf node has between $\lceil (n - 1)/2 \rceil$ and $n - 1$ keys,
- the root is the one exception: it may contain fewer entries than ordinary internal nodes.

These conditions are exactly what keep the tree balanced and keep the height small.

In a B^+ -tree:

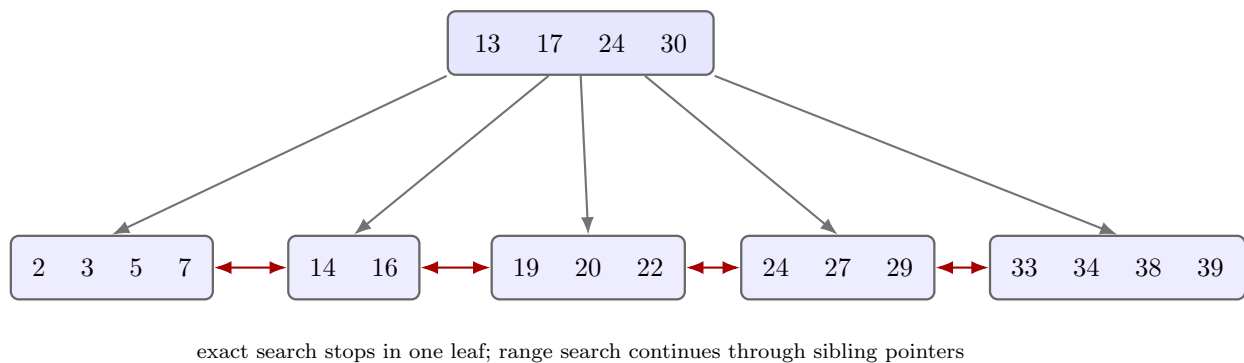


- **internal nodes** store only separator keys and child pointers,
- **leaf nodes** store the actual data entries,
- **sibling pointers** connect leaves from left to right, so after the first match the DBMS can keep scanning sequentially.

The data entry stored in each leaf can still follow Alternative 1, 2, or 3. The tree determines *how to find* the entry; the alternative determines *what the entry contains*.

1.4.3 How search works

Search always starts at the root. Key comparisons choose one child pointer at each internal node until the search reaches a leaf.



The example above shows three typical cases:

Reading searches in a B⁺-tree

Query	Leaf reached	What we learn
find 5	[2, 3, 5, 7]	The key is present in that leaf.
find 15	[14, 16]	The key is not in the tree, because it would have to appear in that leaf between 14 and 16.
find keys ≥ 24	[24, 27, 29]	Start at the first matching leaf, then follow sibling pointers to the right.

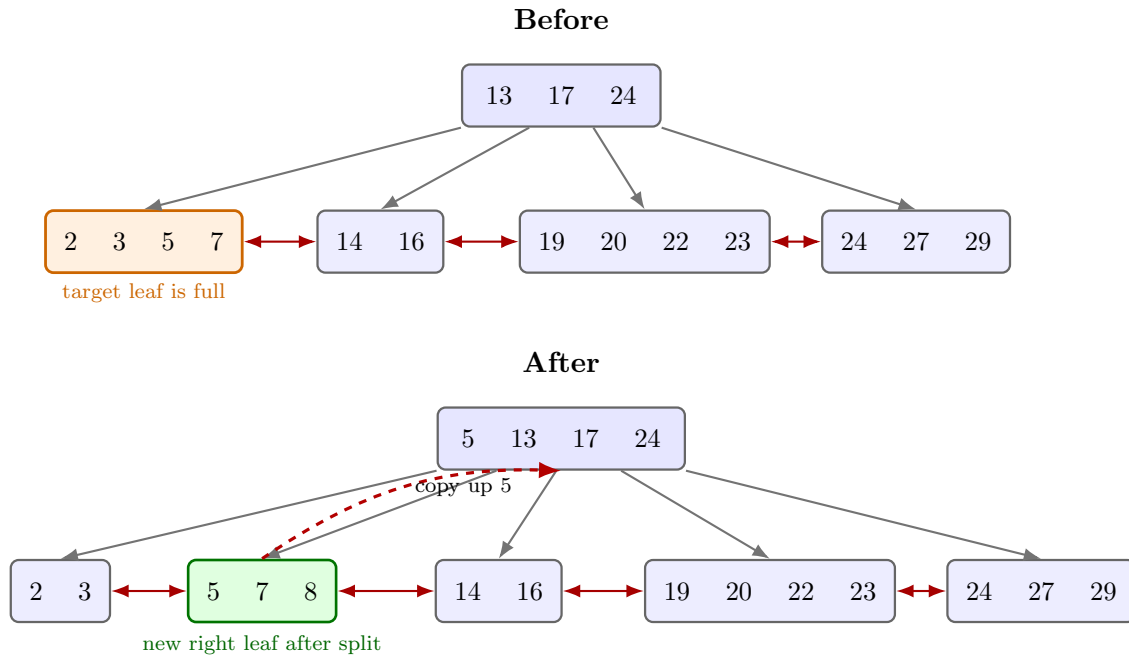
This is why B⁺-trees support both:

- **equality search:** descend to one leaf and check it,
- **range search:** descend once, then scan leaf pages in order.

1.4.4 Insertion example: insert 8

Leaf split. If the target leaf is full, the DBMS allocates a new leaf and redistributes the entries. Then it inserts a separator key into the parent. If the parent also overflows, the split propagates upward.

Suppose we insert key 8 into the tree shown below.



The steps are:

- search from the root and reach the first leaf, because $8 < 13$,
- the target leaf already contains $[2, 3, 5, 7]$, so it is full,
- allocate one new leaf and redistribute the values evenly,
- after inserting 8, the two leaves become $[2, 3]$ and $[5, 7, 8]$,
- copy separator key 5 into the parent, because 5 is now the first key of the new right leaf,
- update sibling pointers so leaf-level scans still work left to right.

In this example, the parent still has room, so the insertion stops there. If the parent were also full, the same split idea would continue upward, possibly all the way to the root.

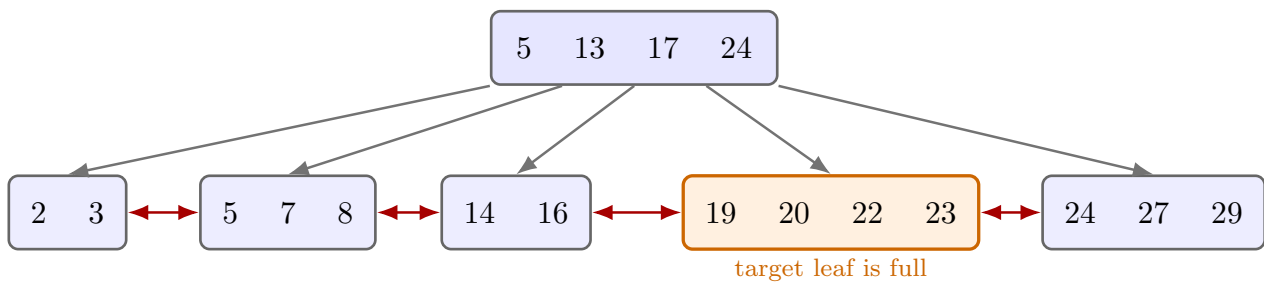
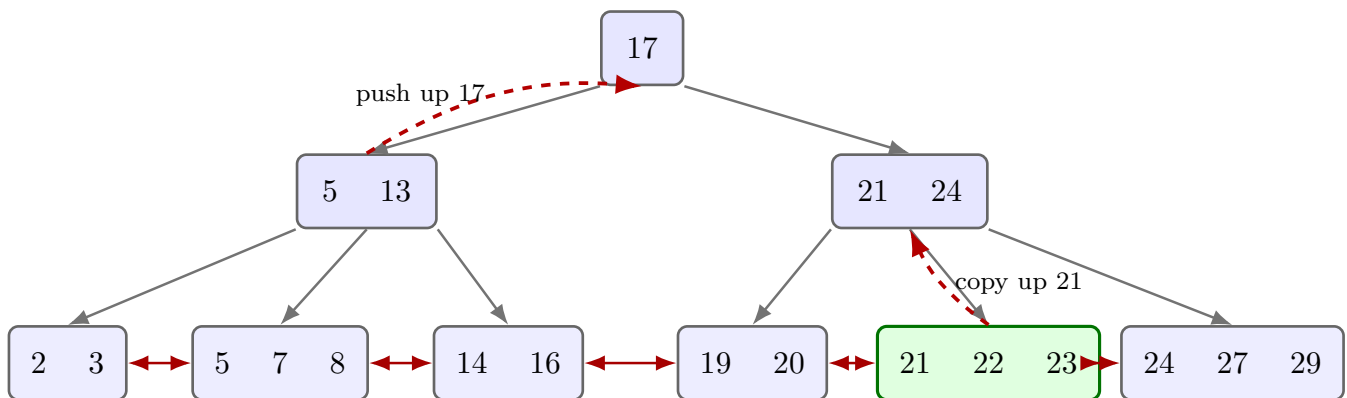
1.4.5 Insertion can propagate to the root

Root split. If an insertion reaches an overflowing root, the DBMS splits the root and creates a new root above it. This is the only case where the height of a B^+ -tree increases.

Now insert key 21 into the current tree. The target leaf is $[19, 20, 22, 23]$, which is already full.

The sequence is:

- insert 21 into the full leaf $[19, 20, 22, 23]$,
- split that leaf into $[19, 20]$ and $[21, 22, 23]$,
- copy separator key 21 into the parent,
- the parent now overflows, so the parent must also split,
- push separator key 17 up to a brand-new root.

Before inserting 21**After split propagation**

The final tree is taller by one level. This is the standard cascading-insertion case: a local split at the leaf causes another split above it.

In theory, redistributing entries differently could delay a root split. In practice, most implementations use the simple split rule because it is local, predictable, and easy to maintain.

1.4.6 Leaf split versus internal split

One detail is especially important: **leaf splits** and **internal splits** do *not* propagate keys in the same way.

Leaf split and internal split

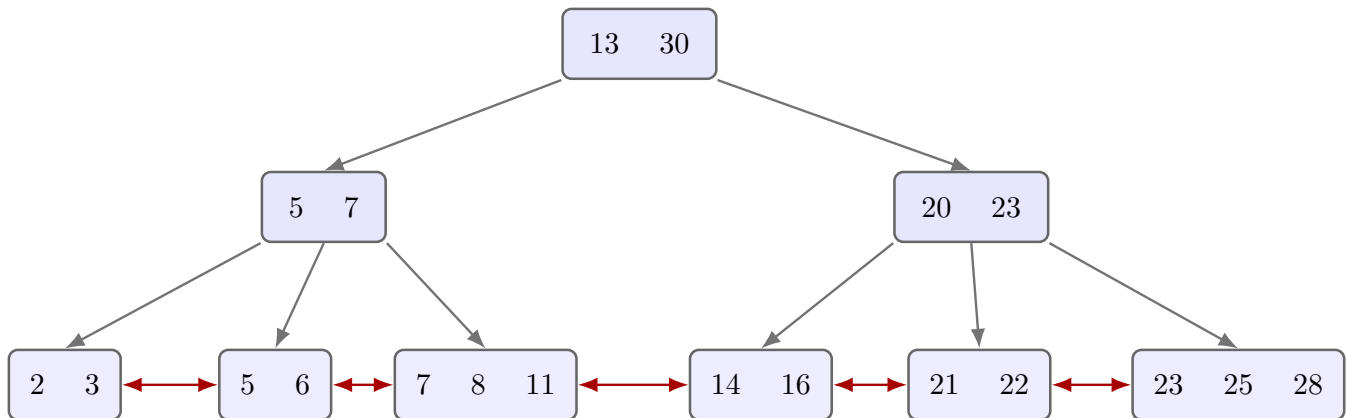
Case	What moves upward	Reason
Leaf split	separator key is copied up	The leaf stores actual data entries, so the key must remain in the leaf. The parent only needs a separator.
Internal split	middle separator is pushed up	Internal-node keys are only routing information, so the promoted separator appears only once, in the parent.

This is the clean rule to remember:

- leaf split $\Rightarrow$ copy up,
- internal split $\Rightarrow$ push up.

1.4.7 Another insertion sequence: 28, 6, and 25

The next example shows three insertions in a row. Some fit locally. Others cause a split and change the upper levels as well.



final tree after inserting 28, 6, and 25

Read the sequence like this:

- **insert 28**: the key fits into the rightmost leaf, so no split is needed,
- **insert 6**: the leaf [5, 7, 8, 11] overflows, so it splits into [5, 6] and [7, 8, 11]; key 7 is copied up,
- **insert 25**: the leaf [21, 22, 23, 28] overflows, so it splits into [21, 22] and [23, 25, 28]; key 23 is copied up,
- the parent then overflows, so an internal split pushes key 13 up and creates the final two-level tree shown above.

This example reinforces the main idea: insertions start locally at one leaf, but the structural effect may travel upward.

1.5 Deletion in B⁺-trees

Deletion in a B⁺-tree. Deletion removes a key from its leaf. If the leaf still satisfies the minimum-occupancy rule, the job is done. Otherwise the DBMS rebalances the tree by redistribution or merge.

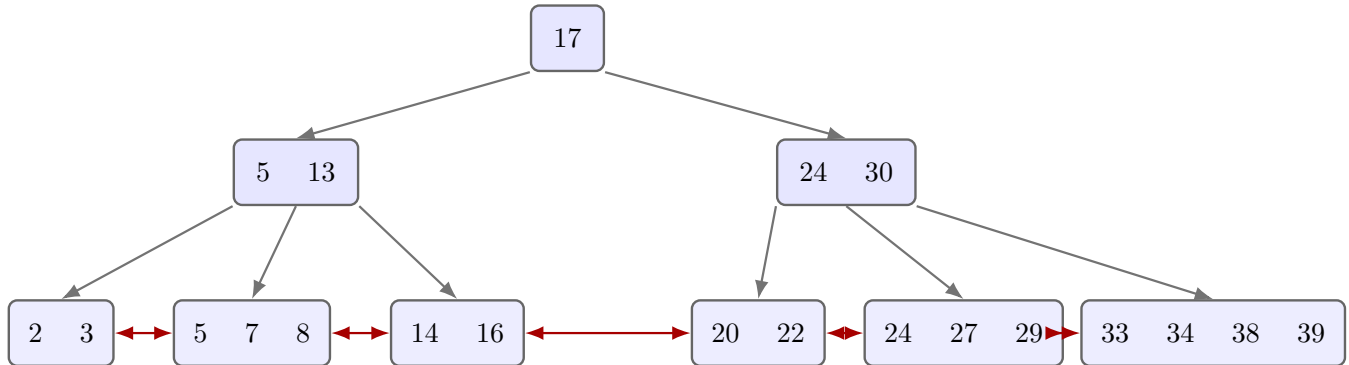
The standard deletion pattern is:

- start at the root and find the leaf L containing the entry,
- remove the entry from L ,
- if L is still at least half full, stop,
- otherwise try **redistribution**: borrow an entry from a sibling with the same parent,
- if no sibling can spare an entry, **merge** L with one sibling,
- after a merge, delete the corresponding separator entry from the parent,
- if the parent underflows, apply the same logic again higher up,
- if the root ends with only one child, remove that root and decrease the tree height.

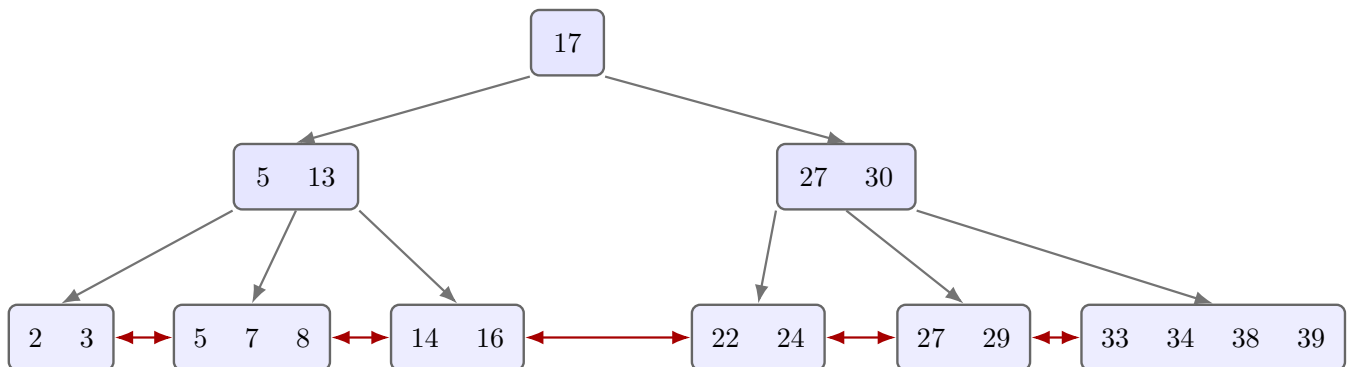
1.5.1 Worked example: delete 19 and 20

Deleting 19 is easy: after removing it, the leaf still has enough entries. Deleting 20 is more interesting, because the leaf becomes too small.

After deleting 19



After deleting 20 and redistributing



After deleting 19, the leaf becomes [20, 22], which is still legal. After deleting 20, the same leaf becomes [22], which violates minimum occupancy.

The DBMS therefore redistributes with the right sibling:

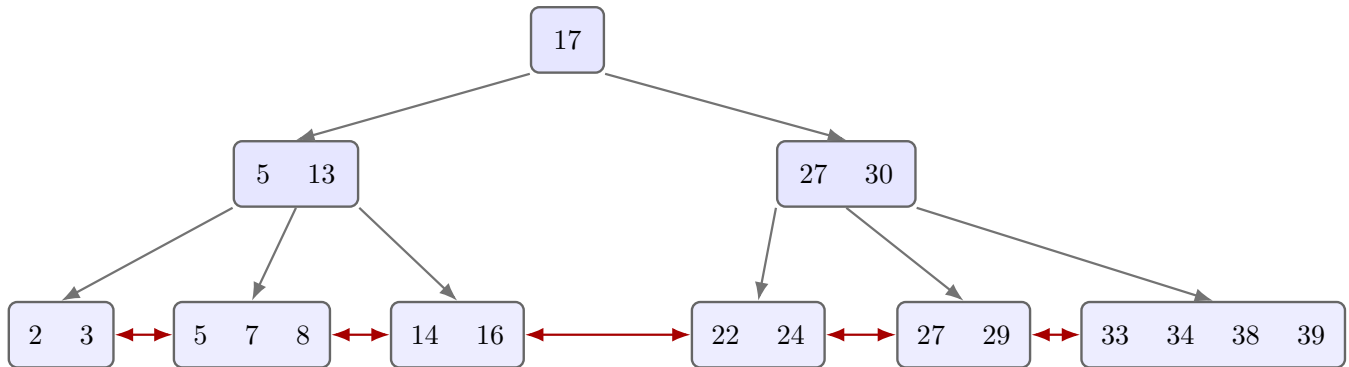
- the right sibling [24, 27, 29] can spare one entry,
- the boundary moves, so the two leaves become [22, 24] and [27, 29],
- the parent separator is updated from 24 to 27.

In a leaf-level redistribution, the new boundary key is **copied up** to the parent, because leaves keep the actual data entries.

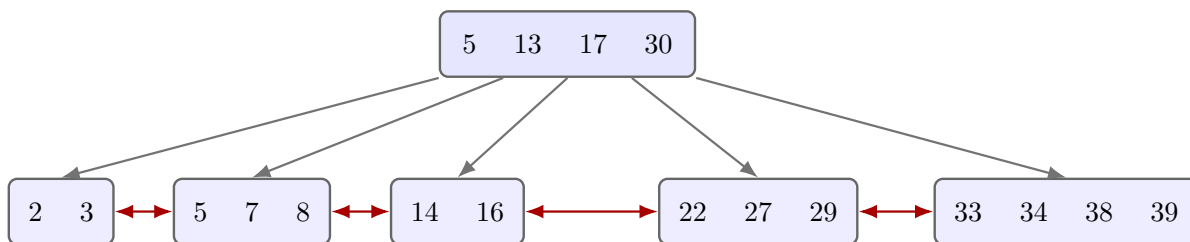
1.5.2 Worked example: delete 24 and shrink the tree

Now continue from the tree above and delete 24. This time redistribution is no longer enough, so the tree must merge and then shrink.

Before deleting 24



After merge and tree shrink



The logic is:

- deleting 24 turns leaf $[22, 24]$ into $[22]$, which is too small,
- its sibling $[27, 29]$ is already at minimum occupancy, so borrowing is impossible,
- the two leaves merge into one leaf $[22, 27, 29]$,
- the parent loses one separator and now underflows,
- the merge therefore propagates upward,
- the old root is removed, and the tree height decreases by one.

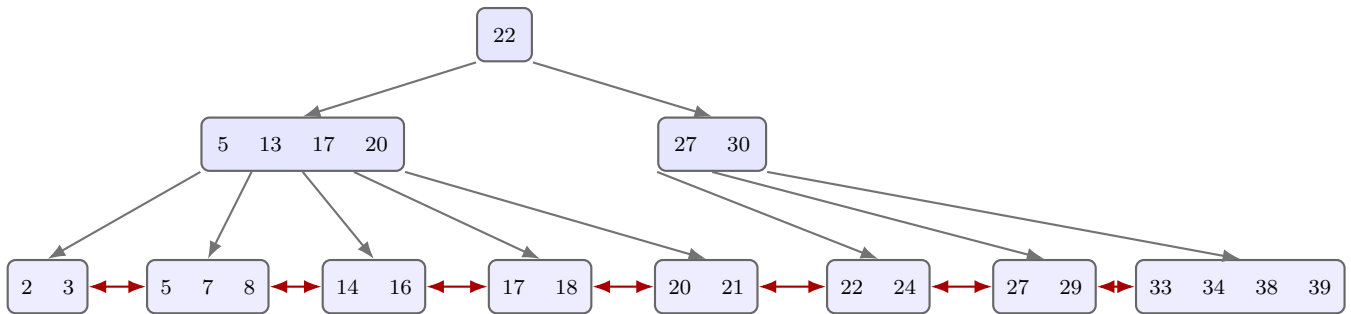
So deletion can do the opposite of insertion:

- insertion may **increase** the tree height,
- deletion may **decrease** the tree height.

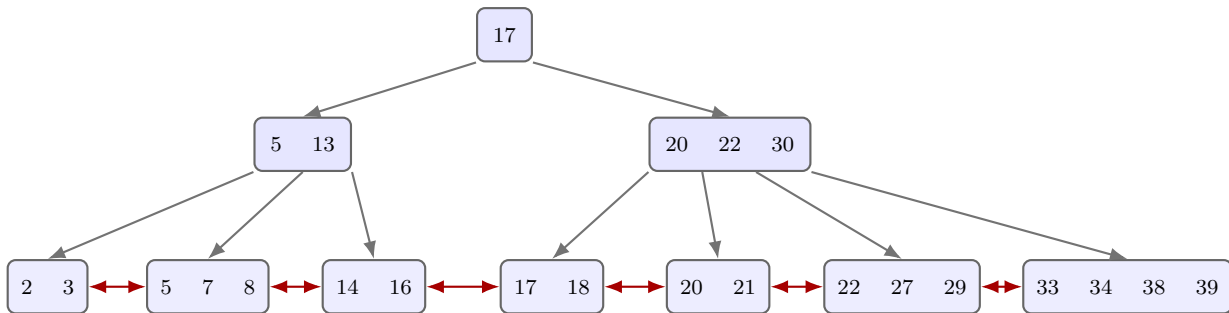
1.5.3 Redistribution at non-leaf levels

Redistribution is not only for leaves. It also works for internal nodes.

Before non-leaf redistribution



After pushing entries through the parent



The key idea is subtle but important:

- redistribution at the non-leaf level happens **through the parent**,
- one separator from the parent moves down into the underfull internal node,
- at the same time, one separator from the fuller sibling moves up into the parent,
- the corresponding child pointer moves with that separator.

So internal-node redistribution is not just “move one key sideways”. It is really a **push-through-parent** operation.

1.6 Clustered indexes

Clustered index. An index whose logical order matches the physical order of the table data. Nearby index entries point to nearby records on disk.

The main idea is physical:

- the table is stored in the sort order of one chosen key, often the primary key,
- the storage may be **heap-organized but kept in that order**, or **index-organized**, where the leaf level stores the actual rows,
- because the data file can have only one physical order at a time, a table can have at most one clustered organization.

This is why clustered indexes are powerful for range scans:

- matching rows tend to sit on nearby pages,
- ordered scans can continue page by page with little random I/O.

But there is also a cost:

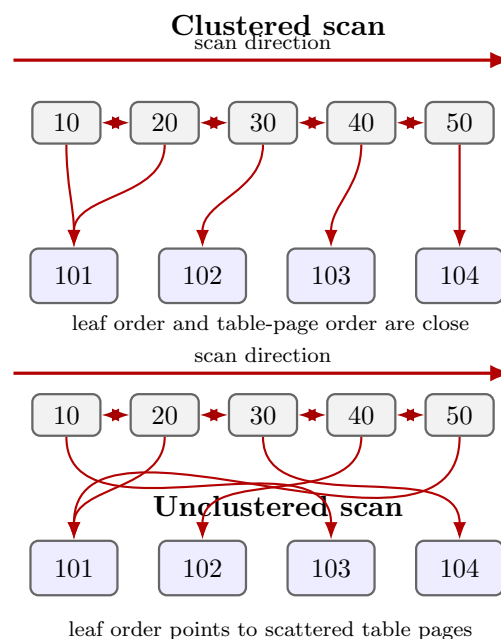
- inserts and deletes are harder, because maintaining physical order may force page splits, movement, or reorganization.

DBMSs differ here:

- some systems always organize a table around a clustered primary key; if no such key is declared, they create a hidden row identifier,
- other systems keep ordinary heap files and do not make the table itself follow one index order.

1.6.1 B⁺-tree traversal and scan order

To scan many records in key order, the DBMS first descends to the leftmost relevant leaf and then follows leaf links from left to right.



The traversal itself is the same in both cases. The big difference is how the leaf entries map to table pages:

- with a **clustered** index, consecutive leaf entries usually point to nearby table pages, so scanning leaf after leaf also reads data pages in a mostly sequential order,
- with an **unclustered** index, consecutive leaf entries may point to tuples scattered over the table, so

following the leaf order can trigger many repeated or random page reads.

This explains why clustered tree indexes are excellent for ordered scans and large range queries.

For unclustered indexes, a common optimization is:

- first collect the matching RIDs or page IDs from the leaves,
- then sort or group them by **table page ID**,
- finally fetch table pages in page order.

This loses the natural key order during fetching, but it can save a lot of I/O when many tuples qualify.

1.7 B⁺-tree design choices

Real systems all use the same core B⁺-tree idea, but they still make many low-level design choices.

1.7.1 Node size

Node size is one of the most important choices because it controls both:

- **fanout**: larger nodes make the tree shallower,
- **intra-node cost**: larger nodes take longer to search once loaded.

As a rule of thumb, the slower the storage device, the larger the best node size tends to be:

Typical node-size tendencies

Storage medium	Typical size	Why
HDD	~ 1 MB	Large transfers amortize expensive seeks and rotational delay.
SSD	~ 10 KB	Random access is cheaper, so extremely large pages are less necessary.
In-memory	~ 512 B	Cache behavior matters more than disk I/O, so smaller nodes can be better.

These numbers are only rough guidelines. The best size still depends on the workload:

- long leaf scans benefit from larger nodes,
- repeated root-to-leaf lookups may prefer smaller nodes that fit caches better.

1.7.2 Merge threshold

The textbook rule says that nodes should stay at least half full. In practice, many DBMSs do *not* merge nodes immediately every time occupancy drops below that threshold.

Why delay merging?

- inserts are often more common than deletes, so a sparse page may soon fill again,
- eager merging causes extra page movement and separator updates,
- in practice, the average occupancy is often around 67%,
- some systems prefer periodic rebuilding over aggressive merge-on-delete.

So the theoretical half-full rule is best understood as a **guarantee for the ideal structure**, not always as a strict implementation policy after every update.

1.7.3 Variable-length keys

Variable-length search keys create another design problem: how do we store them inside fixed-size pages efficiently?

Common approaches include:

- **pointer-based keys:** store a pointer to the real attribute value instead of storing the full key inline,
- **variable-length nodes:** let the packed contents of one node vary in size and manage free space carefully,
- **padding:** extend each key to the maximum length of its type, which simplifies code but wastes space,
- **key indirection:** keep a small array of offsets or pointers that map to the actual key/value area inside the node.

The trade-off is the usual one:

- simpler layout usually wastes more space,
- compact layout saves space but makes memory management harder.

1.7.4 Intra-node search

Once one node is in memory, the DBMS still has to find the correct entry *inside* that node.

Searching inside one node

Method	Main idea	When it is attractive
Linear search	scan keys from left to right	good for small nodes; simple; often fast with SIMD/vectorized comparisons
Binary search	repeatedly compare with the middle key	good when nodes are larger and comparisons are cheap
Interpolation search	estimate the position from the key distribution	useful when keys are close to uniformly distributed

For interpolation search, one common estimate is:

$$\text{pos} \approx \text{low} + \frac{(x - \text{arr}[\text{low}])(\text{high} - \text{low})}{\text{arr}[\text{high}] - \text{arr}[\text{low}]},$$

where x is the search key.

The idea is to jump close to the expected location instead of always probing the middle. This works best when key values are spread fairly evenly.

1.8 Implementation View: Nodes and Lookup

1.8.1 What one node stores

At implementation level, a B^+ -tree node is kept as a **sorted array** of entries inside one page.

The exact meaning of the value part depends on the node type:

- in an **internal node**, the value is a child page reference,
- in a **leaf node**, the value is a record pointer, a RID list, or the actual record itself, depending on the chosen data-entry alternative.

Leaf nodes also keep **prev** and **next** links to neighboring leaves. These are what make range scans efficient.

Even though the internal-node layout has one extra rightmost child pointer, the high-level implementation view is still simple:

- every node stores entries in **sorted key order**,
- search inside a node finds the first separator that tells us where to go next.

1.8.2 Lookup operation

The lookup algorithm follows the sorted structure of the tree:

1. Start at the root node.
2. If the current node is an internal node, search inside it for the first separator that determines the correct child.
3. Follow that child pointer to the next level.
4. Repeat until a leaf is reached.
5. In the leaf, either find the exact entry or determine the exact insertion position where the key would belong.

So a lookup can return either:

- the **matching entry**, if the key exists,
- or the **correct leaf position**, if the key does not exist but might be inserted later.

This lower-bound behavior is exactly what also makes insertions and range scans natural.

1.9 Concurrent Access to B⁺-trees

Concurrency is harder in trees than in isolated heap pages because pages depend on one another structurally.

Simple page latching is not enough:

- it protects one page against concurrent modification,
- but a split or merge also changes parent-child relationships,
- so several pages may need to be synchronized together.

Lock coupling (latch crabbing). During a top-down traversal, the thread latches a parent, then latches the child it wants to enter, and only then releases the parent. The traversal therefore keeps a short overlapping chain of latches.

This classical method works well for search and for many simple updates:

- latch the root first,
- latch the next level,
- release the parent,
- continue downward in canonical root-to-leaf order.

Because latches are acquired in a fixed top-down order, ordinary traversals avoid deadlocks.

1.9.1 Why inserts are trickier

Insertion is harder than lookup because a split may propagate upward:

- a leaf split inserts a separator into the parent,
- that parent may also split,
- the split can continue all the way to the root.

This creates a problem for naive lock coupling: by the time the split reaches higher levels, those ancestors may already have been released.

1.9.2 Restart or optimistic coupling

A simple practical solution is:

1. first try the insertion using ordinary lock coupling,
2. if no split propagates through inner nodes, the operation succeeds normally,
3. otherwise release the latches,
4. restart the operation with a more conservative locking strategy that keeps the needed ancestors protected,
5. execute the update safely on the second pass.

This approach has a clear trade-off:

- it greatly reduces concurrency during the restarted operation,
- but the expensive case is rare,
- and the algorithm is much simpler to implement correctly.

1.10 B⁺-trees in Practice

The main practical lesson is that B⁺-trees are usually **very shallow**.

Many textbooks parameterize them with a value d , where an internal node has between d and $2d$ children. If we take a typical example with:

- $d = 100$,
- average fill factor $\approx 67\%$,

then the average fanout is roughly

$$2d \cdot 0.67 = 200 \cdot 0.67 \approx 134.$$

That already gives very large capacities:

Back-of-the-envelope capacities

Tree height	Approximate number of entries
3	$134^3 = 2,406,104$
4	$134^4 = 322,417,936$

This is why a tree of height 3 or 4 is often enough even for very large relations.

Top levels are also small enough to remain in memory:

Example memory footprint with 8 KB pages

Level	Size
level 1	1 page $\approx$ 8 KB
level 2	134 pages $\approx$ 1 MB
level 3	17,956 pages $\approx$ 140 MB

So in practice:

- the root is almost always memory-resident,
- often one or more upper levels are memory-resident too,
- only the lower levels require regular I/O.